



§. 输入输出重定向及管道运算符

0. 第一次编程作业的不方便之处

- ★ 一个作业需要多次测试，每次Ctrl+F5运行后输入，**重复多次**
- ★ 如果作业测试过程中发现问题，修改后又得全部测试一遍，**重复多次**
- ★ 即使VS2026通过了，还得测试小熊猫，**重复多次**
- ★ 输出结果只能肉眼观察是否与demo匹配，**无法知道准确结果**
- ★ 不知道怎么设计测试数据

本文档解决的问题：**快速重复并准确知道结果正确与否**（红框4个痛点）



§. 输入输出重定向及管道运算符

1. 预备知识
2. 输入输出重定向
3. 批处理的基本概念
4. 用输入输出重定向测试程序
5. 用管道运算符测试程序

本文档分别对应5个视频讲解文件，都放在附件中

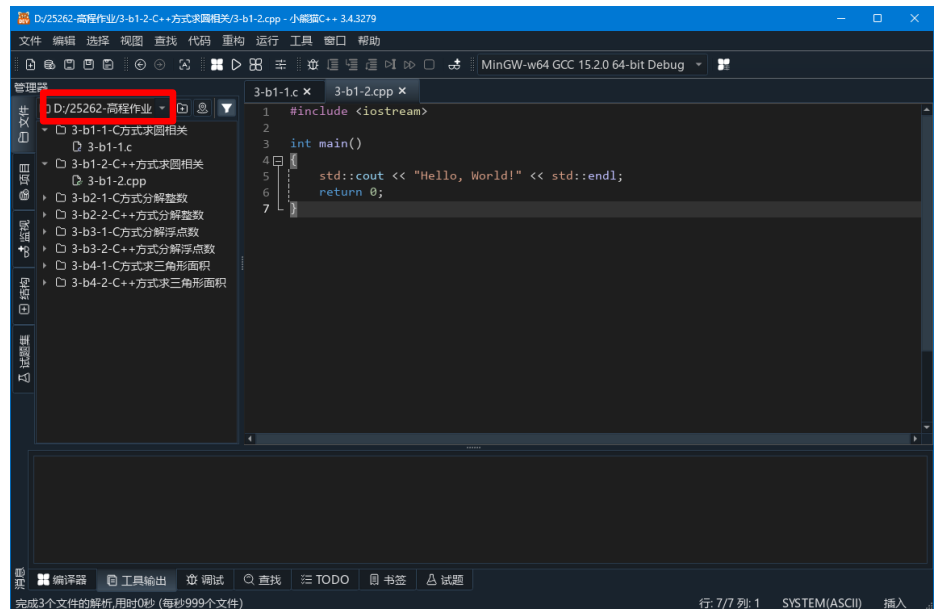
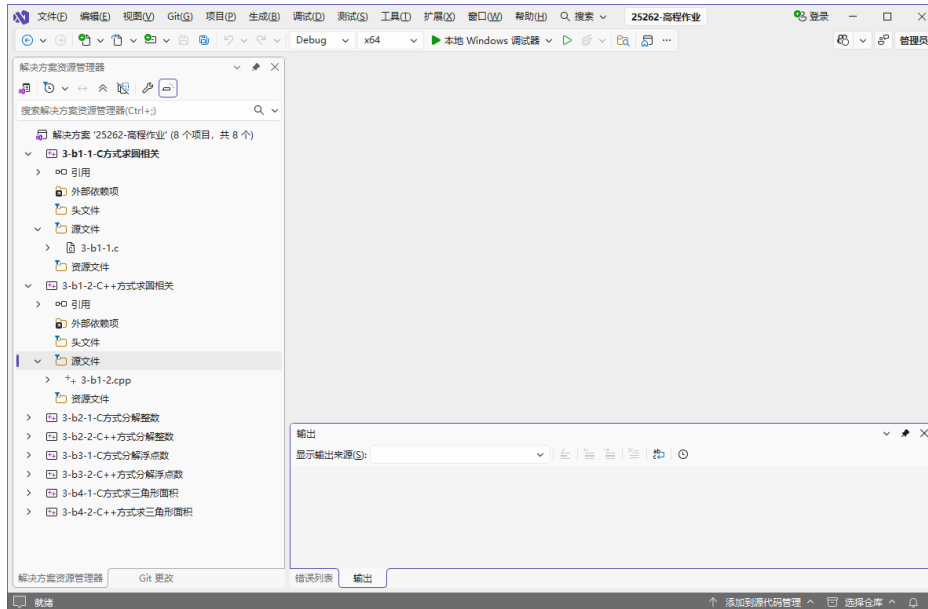


§. 输入输出重定向及管道运算符

1. 预备知识

1.1. 准备工作：以“25262-030103-W0401. 附件 编程作业需要了解的知识点汇总. pdf”中的设置为例

★ D盘根目录下建立“25262-高程作业”解决方案，其中包含本学期所有编程作业，同时将小熊猫C++的工作目录设置到此文件夹下



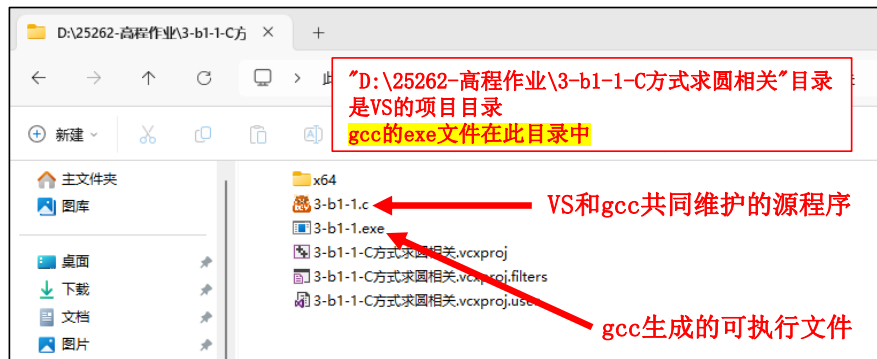
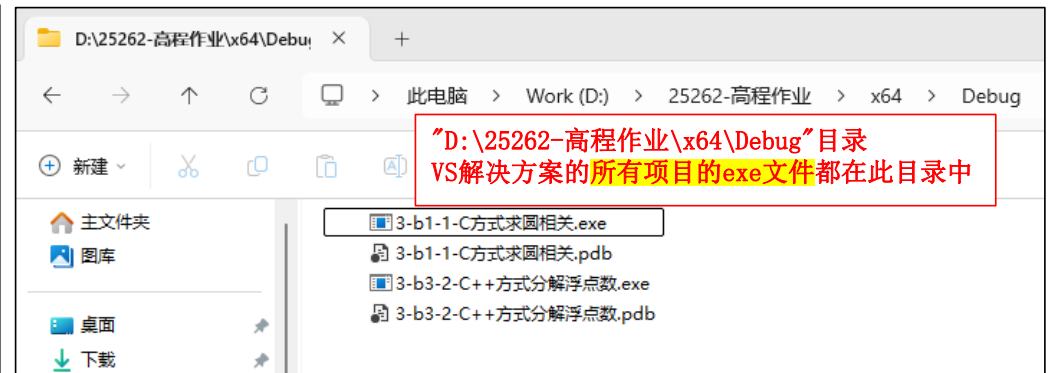
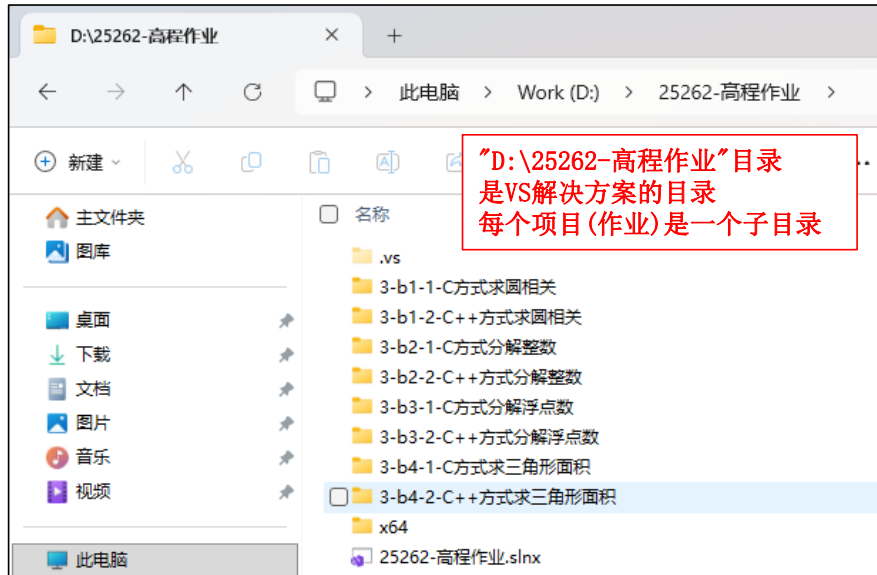
★ 注：各位同学按需在自己需要的位置建立解决方案，并且能够找到对应位置即可



§. 输入输出重定向及管道运算符

1. 预备知识

1. 2. VS2026/gcc编译生成的可执行文件(*.exe)的位置说明



说明:

- 1、本文档后续说明均以此目录结构为例, 实际使用时要对应到自己的相应目录下
- 2、VS2026生成的可执行文件, 在与项目(3-b1-1等)同级的x64/Debug目录中
- 3、gcc生成的可执行文件名, 与源程序(*.cpp)在同一目录下



§. 输入输出重定向及管道运算符

1. 预备知识

1.3. 在图形化界面中双击exe执行导致“闪退”的原因解释

★ 在集成环境下运行与直接双击exe文件名方式运行的差异

- 在集成环境下运行：直接 CTRL+F5 (VS) / F11 (小熊猫) 运行，结束后提示按键关闭
- 直接双击exe文件名运行：“闪退”

★ “闪退”的现象及原因解释

双击exe后，会自动开启一个cmd窗口，随后运行程序，运行完成后立即关闭

★ 解决“闪退”的方法

- 方法1：在程序最后加一句 `system("pause")` / 其它方式的读键盘语句

=> 多人批量运行时，完成时必须按键才能继续下一个人，因此会“卡死”，导致批处理无法自动执行
(仅限于演示，不允许作业中使用，否则0分)

- 方法2：先手动启动一个cmd窗口，在窗口中用命令行方式运行 (1.4~1.6，必须掌握)

★ 从即日起，下发的所有参考exe，均为命令行方式执行

★ 再次强调：提交的作业不要加`system("pause")`等暂停/读键语句，否则会因为检查作业时卡死而被判为死循环，得分为0



§. 输入输出重定向及管道运算符

1. 预备知识

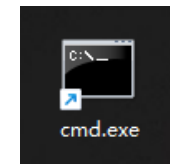
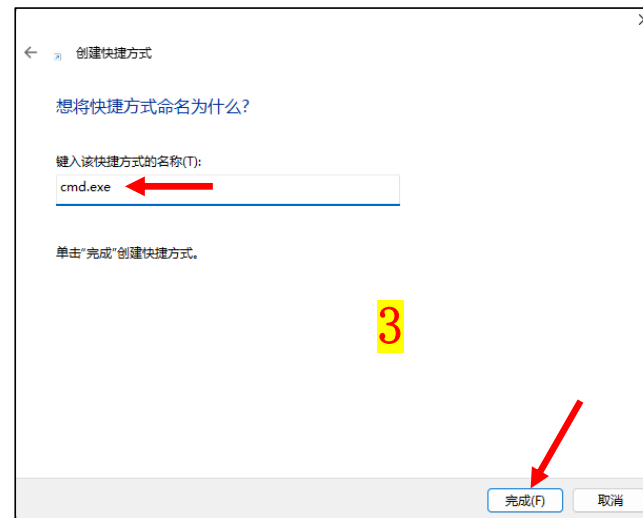
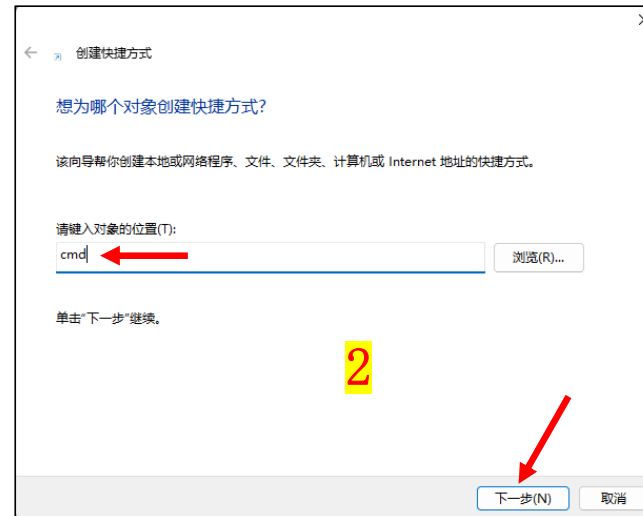
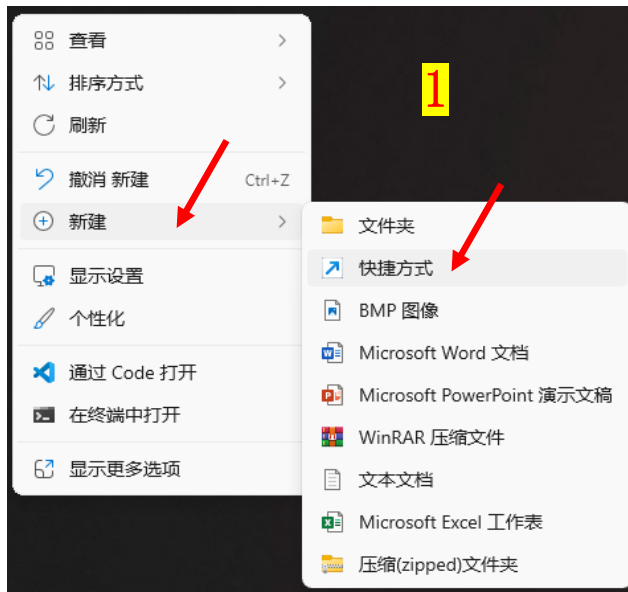
1.4. 在桌面上建立cmd.exe的快捷方式

Step1: 桌面空白处右键 - 新建 - 快捷方式

Step2: 出现的对话框中输入cmd, 按“下一步”

Step3: 输入该快捷方式的名称, 按“完成”

Step4: 桌面出现cmd.exe图标





§. 输入输出重定向及管道运算符

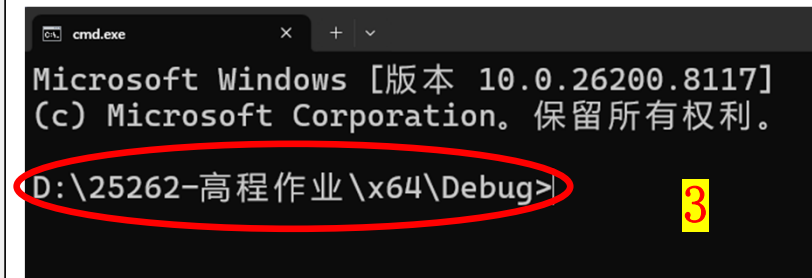
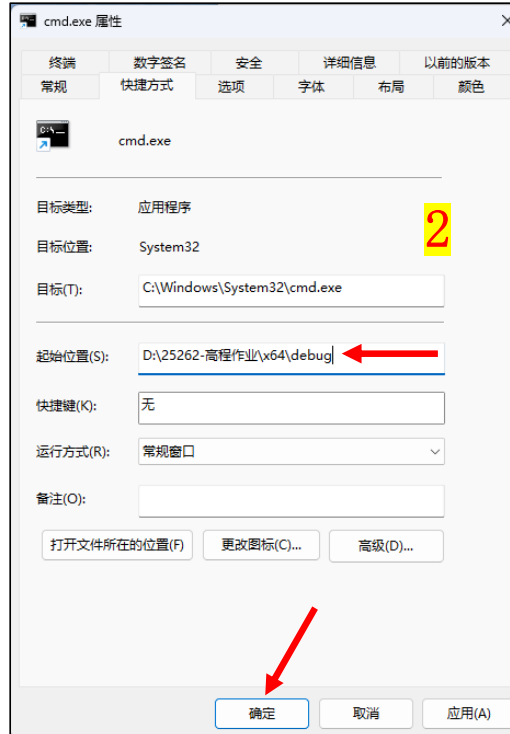
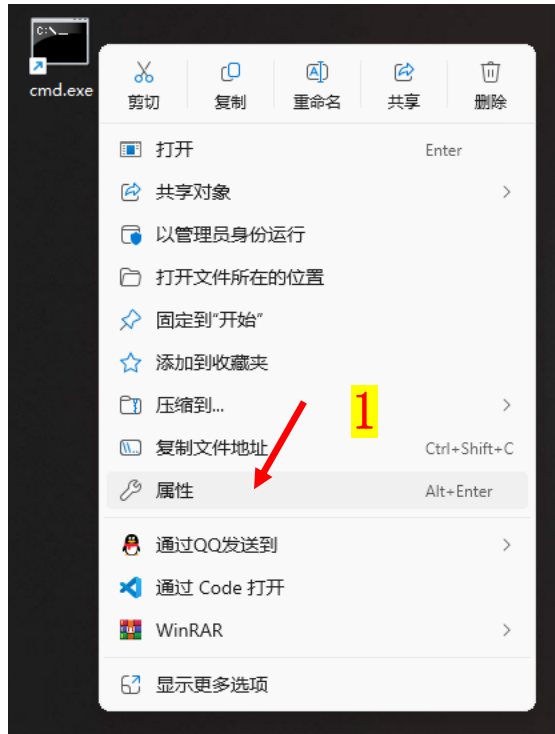
1. 预备知识

1.5. 设置启动cmd.exe快捷方式后缺省进入指定目录(本文档以D:\25262-高程作业\x64\debug为例, 可对应修改)

Step1: 在cmd.exe快捷方式上右键 - 属性

Step2: 出现的对话框中, 将“起始位置”改为指定目录, 按“确定”

step3: 双击cmd.exe快捷方式, 确认进入D:\25262-高程作业\x64\Debug

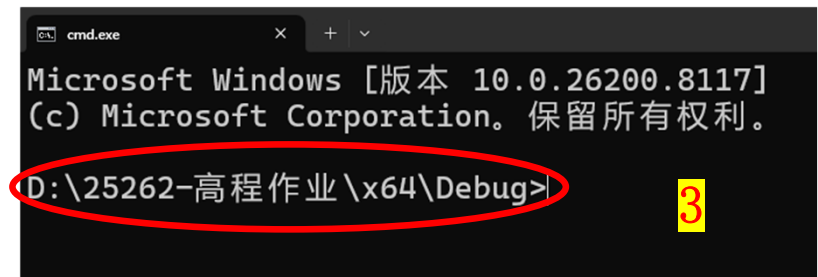
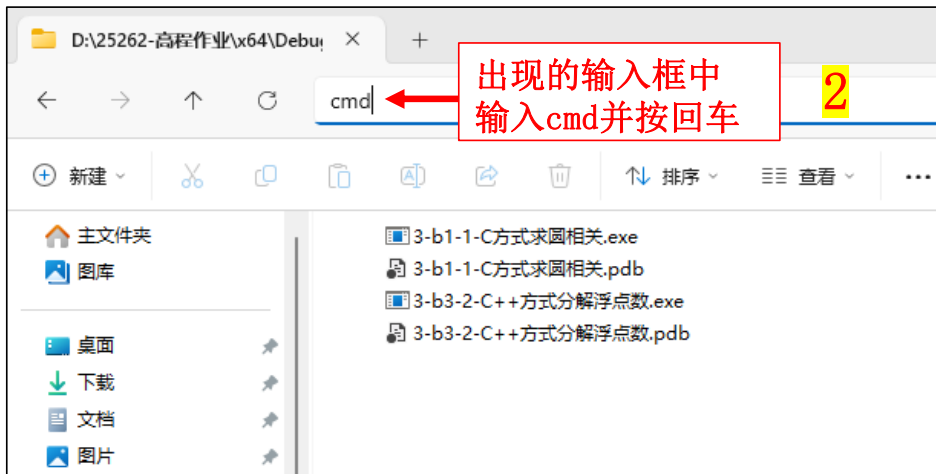
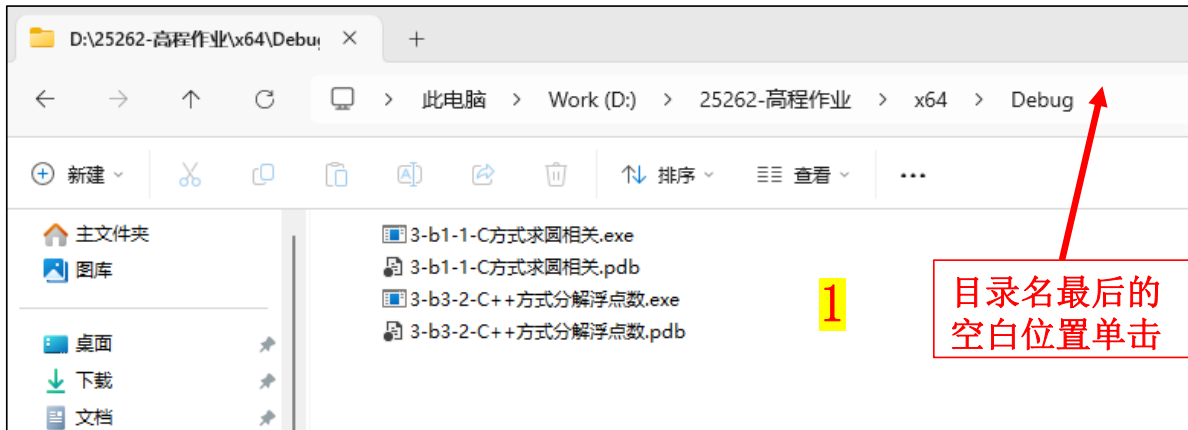




§. 输入输出重定向及管道运算符

1. 预备知识

1.6. 通过文件管理器启动cmd.exe，直接进入对应目录的方法(本文档以D:\25262-高程作业\x64\debug为例，可对应修改)





§. 输入输出重定向及管道运算符

2. 输入输出重定向

2.1. 基本概念

输出重定向：程序执行时，系统**默认的输出设备**是**显示器**，如果改为其他设备/文件，则称为输出重定向

输入重定向：程序执行时，系统**默认的输入设备**是**键盘**，如果改为其他设备/文件，则称为输入重定向



§. 输入输出重定向及管道运算符

2. 输入输出重定向

2.2 将输出重定向到文件中

2.2.1. 输出的分类

cout: 标准输出

cerr: 错误输出

clog: 错误输出

例: 观察下面程序的运行结果（在“25262-高程作业”解决方案中新建一个demo项目）

(1) 集成环境下

(2) cmd窗口中

注: VS和gcc生成的exe在不同目录下, 为方便, 将gcc的exe文件移动到VS的exe文件所在的目录中, 改名为demo.gcc.exe

```
#include <iostream>
using namespace std;

int main()
{
    cout << "标准输出(cout)" << endl;
    cerr << "错误输出(cerr)" << endl;
    clog << "错误输出(clog)" << endl;

    return 0;
}
```



§. 输入输出重定向及管道运算符

2. 输入输出重定向

2.2 将输出重定向到文件中

2.2.2. 标准和错误输出重定向到文件中

★ 将2.1中VS/gcc生成的exe文件在cmd窗口下运行，分别输入不同的命令，观察运行结果

进入cmd窗口，分别输入以下命令，观察运行结果

- 1、[demo](#)
- 2、[demo >a.txt](#)
- 3、[demo 1>a.txt](#)
- 4、[demo 2>a.txt](#)
- 5、[demo 1>a.txt 2>b.txt](#)
- 6、[demo 1>a.txt 2>&1](#)

注：1、cmd下运行exe时，“.exe”后缀可以不加

2、观察运行果时，注意目录下的文件变化，以及a.txt/b.txt等文件中内容的变化

● dir : 用来列出当前目录下的文件

● notepad a.txt : 用“记事本”查看a.txt文件

3、自行替换为demo.gcc.exe，观察结果（应相同）



§. 输入输出重定向及管道运算符

2. 输入输出重定向

2.2. 将输出重定向到文件中

2.2.3. 重定向文件的追加

★ 引入：上例中，若命令反复执行，则a.txt或b.txt的内容仅会保留最后一次

★ 解决：将>换为>>即可不断追加而不清空原有内容

进入cmd窗口，分别输入以下命令，观察运行结果

1、demo

2、demo >a.txt (尝试一次>, 多次>>)

3、demo 1>a.txt (尝试一次>, 多次>>)

4、demo 2>a.txt (尝试一次>, 多次>>)

5、demo 1>a.txt 2>b.txt (尝试一次>, 多次>>)

6、demo 1>a.txt 2>&1 (尝试一次>, 多次>>)

注：1、cmd下运行exe时，".exe"后缀可以不加

2、观察运行果时，注意目录下的文件变化，以及a.txt/b.txt等文件中内容的变化

● dir : 用来列出当前目录下的文件

● notepad a.txt : 用"记事本"查看a.txt文件

3、自行替换为demo.gcc.exe，观察结果（应相同）



§. 输入输出重定向及管道运算符

2. 输入输出重定向

2. 2. 将输出重定向到文件中

2. 2. 4. 在图形化界面中双击exe执行导致“闪退”的原因解释

见1. 3



§. 输入输出重定向及管道运算符

2. 输入输出重定向

2.3. 将输入重定向为来自文件中

例：观察下面程序的运行结果

(1) 集成环境下

(2) cmd窗口中

注：VS和gcc生成的exe在不同目录下，为方便，将gcc的exe文件移动到VS的exe文件所在的目录中，改名为demo.gcc.exe

```
#include <iostream>
using namespace std;

int main()
{
    int a, b;

    cout << "请输入两个整数" << endl;
    cin >> a >> b;
    cerr << "a=" << a << " b=" << b << endl;
    cout << "大数是：" << (a>b?a:b) << endl;

    return 0;
}
```

★ 建立**纯文本文件**z.dat，写入两个整数，再保存

方法1：所在文件夹的空白处鼠标右键 - 新建 - 文本文档，再改名

方法2：cmd窗口中输入 notepad z.dat

D:\25262-高程作业\x64\Debug>notepad z.dat

注：其他软件（VSCode等）也能编辑，但一定要是**纯文本文件**

★ 进入cmd窗口，分别输入以下命令，观察运行结果

1、demo

2、demo < z.dat

问题：

如果z.dat中 (1) 仅有1个整数

(2) 3个及以上整数

(3) 不是整数（类似于12a34这种）

(4) 不是整数（字母或符号开头）

demo < z.dat：运行结果？



§. 输入输出重定向及管道运算符

2. 输入输出重定向

2.4. 同时进行输入/输出重定向

★ 上例的exe，将命令组合即可（顺序任意），观察运行结果

```
demo <z.dat 1>a.txt
```

```
demo 1>a.txt <z.dat
```

```
demo >a.txt <z.dat
```

```
demo 1>a.txt 2>b.txt <z.dat
```

```
demo 1>a.txt 2>&1 <z.dat
```

● 也可以自行将>换成>>，观察结果



§. 输入输出重定向及管道运算符

3. 批处理的基本概念

★ 基本概念：“批量”进行处理，提高效率

- 降低重复输入命令时的出错概率
- 可以重复运行
- 简单修改后就可以适应其他作业

★ Windows下批处理文件的建立

● 方法1:

- 1、所在文件夹的空白处鼠标右键 - 新建 - 文本文档
- 2、修改主文件名并将文件名后缀改为.bat（也可以是.cmd）
- 3、该bat文件上鼠标右键 - 在记事本中编辑

● 方法2:

在cmd窗口中直接 `notepad xx.bat` 可以直接新建/打开bat文件供编辑/修改（一句命令替代步骤1-3）

★ 批处理文件的内容

- 能够在cmd窗口中正确执行的命名
- 用于控制批处理执行的一些特殊命令（仅介绍rem和@echo off，其他自行了解）

★ 批处理文件的执行

- 在cmd下输入文件名执行（需要查看结果）
- 鼠标双击执行（只需要执行结果，如果想看运行提示，最后加 `pause`）

★ 特别注意：如果批处理文件中含有中文，一定要另存为ANSI/GB18030



§. 输入输出重定向及管道运算符

- 4. 用输入/输出重定向测试程序(以浮点数分解3-b3-2为例)
 - 4.1. 准备测试数据
 - 4.2 单数据调试
 - 4.3 单数据测试并和demo对比运行结果
 - 4.4 多数据测试并和demo对比运行结果



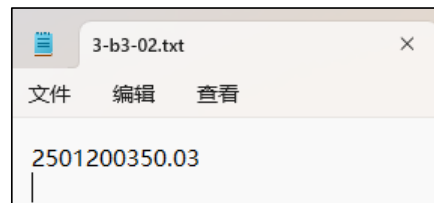
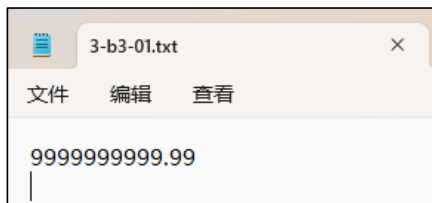
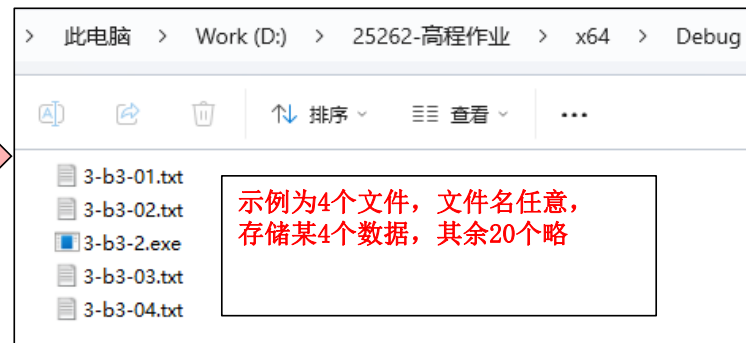
§. 输入输出重定向及管道运算符

4. 用输入/输出重定向测试程序(以浮点数分解3-b3-2为例)

4.1. 准备测试数据

★ 在D:\25262-高程作业\x64\debug(本文档示例目录, 自行替换)下建立24个**纯文本文件**, 将要求的24个测试数据分别放入, 每个文件一个数据

9999999999.99	9999999999.90	9999999999.09	9900000000
8912003005.78	2501200350.03	1203056740.00	203056740.20
23000056.82	3051200.72	301000.35	10001.34
8070.23	9876.54	803.03	12.30
10.03	9.30	7.03	0.35
0.30	0.07	0.03	0





§. 输入输出重定向及管道运算符

4. 用输入/输出重定向测试程序(以浮点数分解3-b3-2为例)

4.1. 准备测试数据

★ 在D:\25262-高程作业\x64\debug(本文档示例目录, 自行替换)下建立24个**纯文本文件**, 将要求的24个测试数据分别放入, 每个文件一个数据

★ 将作业附件给出的3-b3-demo.exe也放在D:\25262-高程作业\x64\debug(本文档示例目录, 自行替换)下, **同时用**输入输出重定向方式建立参考答案文件**(只需要做一次, 也可以放入批处理文件中)**

```
cmd.exe
Microsoft Windows [版本 10.0.26200.8117]
(c) Microsoft Corporation. 保留所有权利。

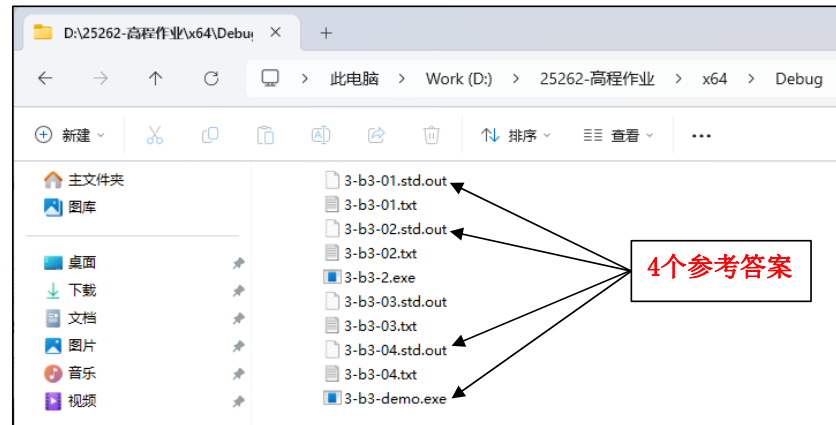
D:\25262-高程作业\x64\Debug>make-3-b3_std_ans.bat

D:\25262-高程作业\x64\Debug>3-b3-demo.exe 0<3-b3-01.txt 1>3-b3-01.std.out
D:\25262-高程作业\x64\Debug>3-b3-demo.exe 0<3-b3-02.txt 1>3-b3-02.std.out
D:\25262-高程作业\x64\Debug>3-b3-demo.exe 0<3-b3-03.txt 1>3-b3-03.std.out
D:\25262-高程作业\x64\Debug>3-b3-demo.exe 0<3-b3-04.txt 1>3-b3-04.std.out
D:\25262-高程作业\x64\Debug>
```

批量运行4次,
生成4个参考答案文件

```
make-3-b3_std_ans.bat
文件 编辑 查看

3-b3-demo.exe < 3-b3-01.txt > 3-b3-01.std.out
3-b3-demo.exe < 3-b3-02.txt > 3-b3-02.std.out
3-b3-demo.exe < 3-b3-03.txt > 3-b3-03.std.out
3-b3-demo.exe < 3-b3-04.txt > 3-b3-04.std.out
```



注: 输出文件的名字可以任意指定, 建议**自行制订**一套命名规则, 方便管理

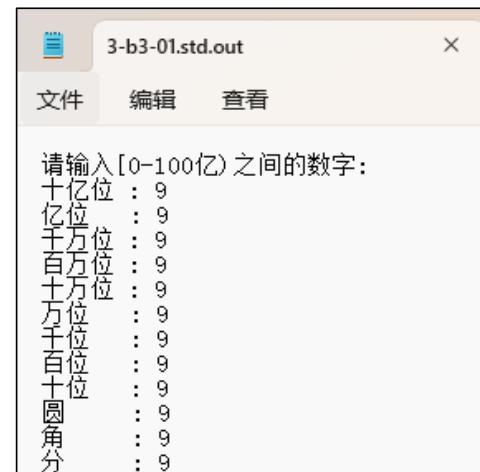
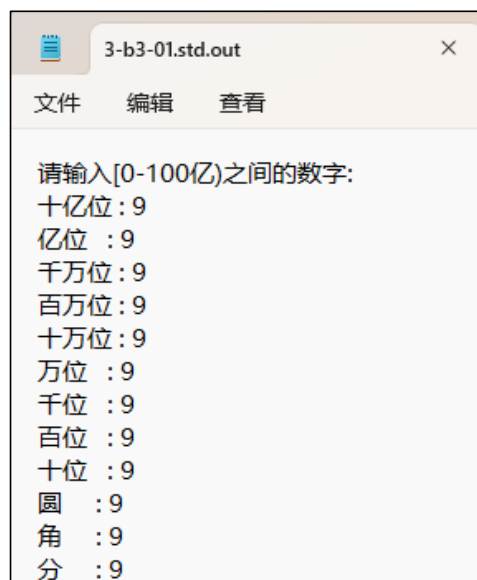
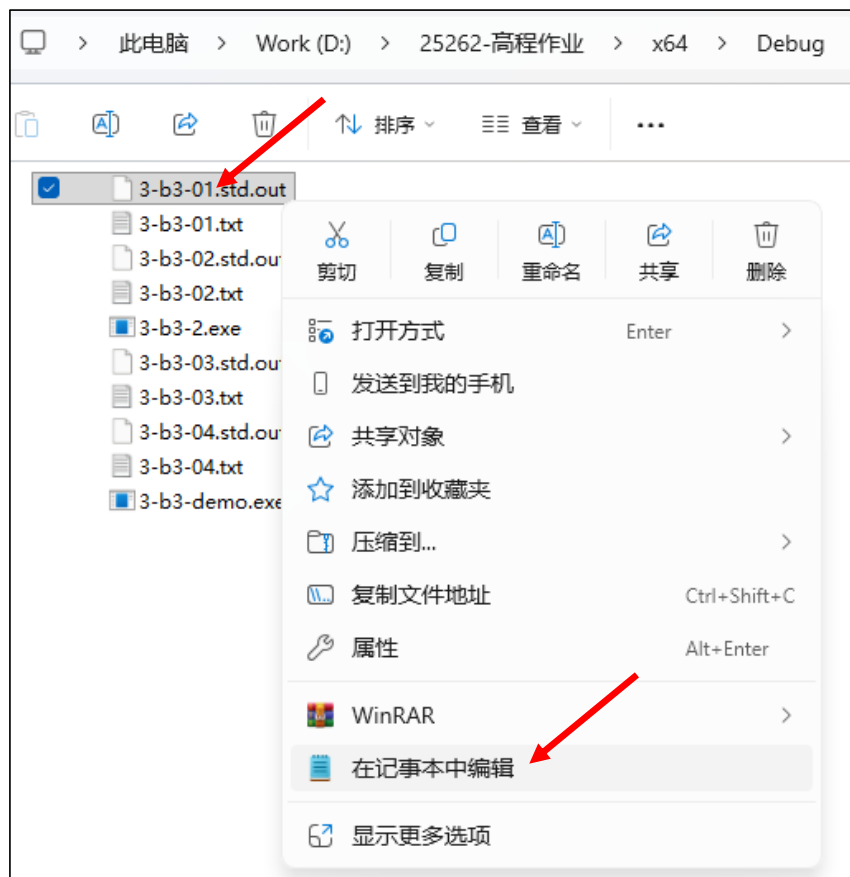


§. 输入输出重定向及管道运算符

4. 用输入/输出重定向测试程序(以浮点数分解3-b3-2为例)

4.1. 准备测试数据

★ 用记事本打开输出文件后**字符显示对不齐**问题的解决（设置等宽字体：例如宋体等）





§. 输入输出重定向及管道运算符

4. 用输入/输出重定向测试程序(以浮点数分解3-b3-2为例)

4.2 单数据调试

★ 编程初期, 程序还有很多错误的情况下, 在VS的集成环境下调试并运行

★ 将生成的3-b3-2.exe文件在cmd下用键盘输入/输入重定向方式运行, 肉眼观察屏幕输出

注: 在cmd窗口中输入可执行文件名时, .exe可以不要

```
D:\25262-高程作业\x64\Debug>3-b3-2
请输入[0-100亿)之间的数字:
9999999999.99
十亿位 : 9
亿位 : 9
千万位 : 9
百万位 : 9
十万位 : 9
万位 : 9
千位 : 9
百位 : 9
十位 : 9
圆 : 9
角 : 9
分 : 9
D:\25262-高程作业\x64\Debug>
```

键盘输入方式
此句是键盘输入

```
D:\25262-高程作业\x64\Debug>3-b3-2 < 3-b3-01.txt
请输入[0-100亿)之间的数字:
十亿位 : 9
亿位 : 9
千万位 : 9
百万位 : 9
十万位 : 9
万位 : 9
千位 : 9
百位 : 9
十位 : 9
圆 : 9
角 : 9
分 : 9
D:\25262-高程作业\x64\Debug>
```

输入重定向方式
无键盘输入



§. 输入输出重定向及管道运算符

4. 用输入/输出重定向测试程序(以浮点数分解3-b3-2为例)

4.3 单数据测试并和demo对比运行结果

★ 将生成的3-b3-2.exe在cmd下**同时用**输入输出重定向方式运行，得到**重定向输出文件**

注：1、输出文件的名字可以任意指定，建议**自行制订**一套命名规则，方便管理

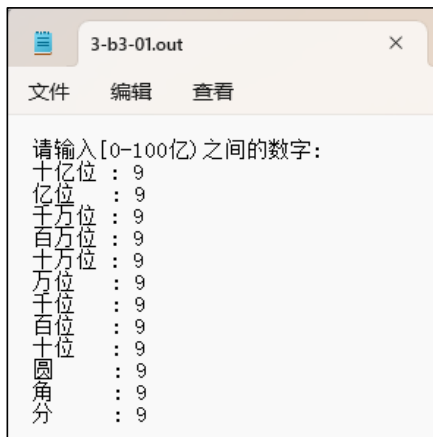
2、可以用批处理命令提高效率

```
D:\25262-高程作业\x64\Debug>notepad 3-b3-test.bat
```



```
D:\25262-高程作业\x64\Debug>3-b3-test.bat
```

```
D:\25262-高程作业\x64\Debug>3-b3-2.exe 0<3-b3-01.txt 1>3-b3-01.out
```





§. 输入输出重定向及管道运算符

4. 用输入/输出重定向测试程序(以浮点数分解3-b3-2为例)

4.3 单数据测试并和demo对比运行结果

★ 将生成的3-b3-2.exe在cmd下**同时用**输入输出重定向方式运行，得到**重定向输出文件**

★ 用txt_compare工具对输出文件进行比对，确实是否完全一致

- txt_compare的使用方法：在cmd下运行txt_compare后查看帮助信息
- txt_compare的必选参数：--file1 xxx -file2 xxx
- txt_compare的常用可选参数：--display normal/detailed
--trim right
- 目标：

```
D:\25262-高程作业\x64\Debug>txt_compare --file1 3-b3-01.out --file2 3-b3-01.std.out  
文件相同。
```



§. 输入输出重定向及管道运算符

4. 用输入/输出重定向测试程序(以浮点数分解3-b3-2为例)

4.3 单数据测试并和demo对比运行结果

★ 将生成的3-b3-2.exe在cmd下**同时用**输入输出重定向方式运行，得到**重定向输出文件**

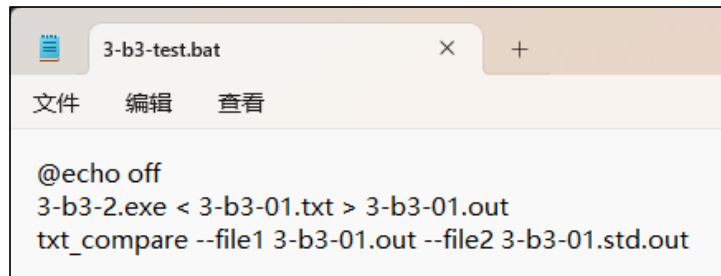
★ 用txt_compare工具对输出文件进行比对，确实是否完全一致

- txt_compare的使用方法：在cmd下运行txt_compare后查看帮助信息
- txt_compare的必选参数：--file1 xxx -file2 xxx
- txt_compare的常用可选参数：--display normal/detailed
--trim right
- 目标：

```
D:\25262-高程作业\x64\Debug>txt_compare --file1 3-b3-01.out --file2 3-b3-01.std.out  
文件相同。
```

★ **提高效率的方法**：将生成输出文件和用txt_compare比对的命令依次放入批处理文件中

```
D:\25262-高程作业\x64\Debug>notepad 3-b3-test.bat
```

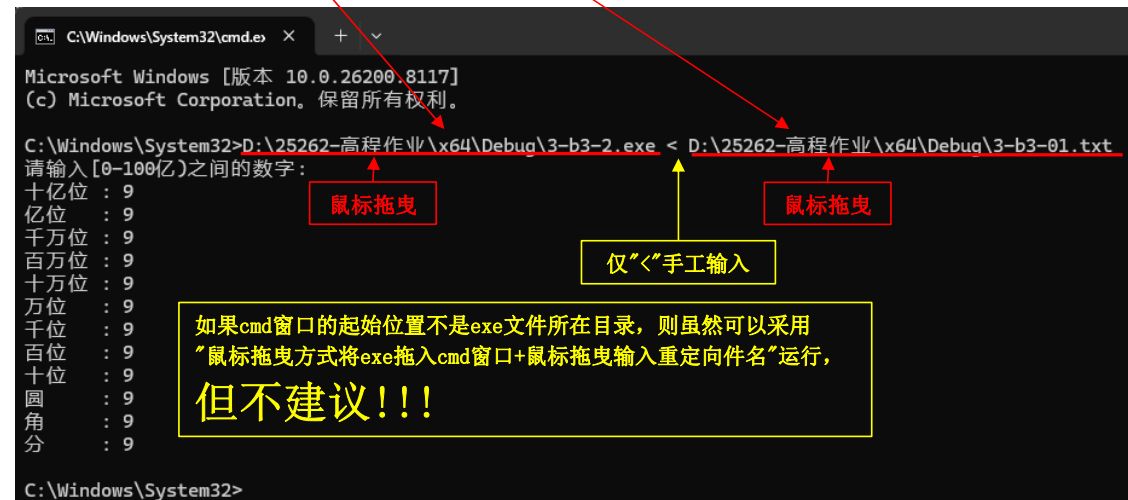
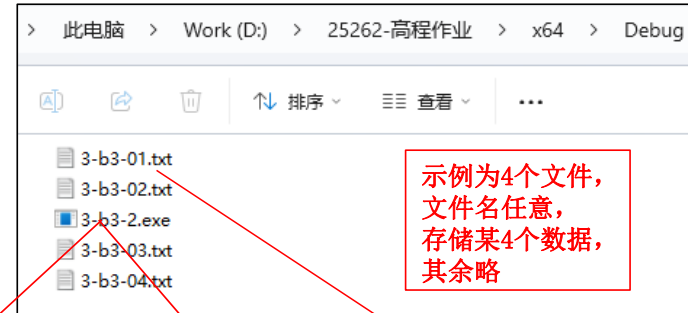
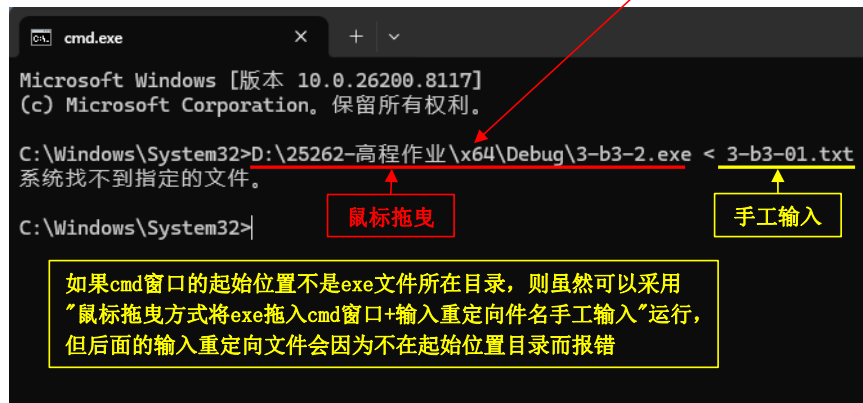
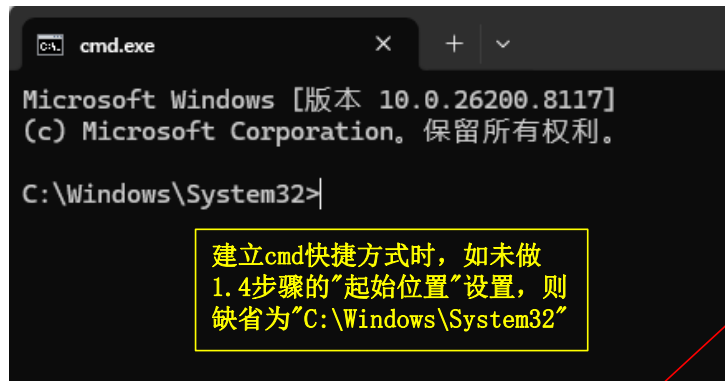


```
D:\25262-高程作业\x64\Debug>3-b3-test.bat  
文件相同。
```




§. 输入输出重定向及管道运算符

错误和不建议的用法:



另: 后续的作业会给出demo.exe, 如果要求exe在cmd下运行, 同样不建议用鼠标拖曳!!!



§. 输入输出重定向及管道运算符

4. 用输入/输出重定向测试程序(以浮点数分解3-b3-2为例)

4.4 多数据测试并和demo对比运行结果

★ 将多个测试点放入同一个批处理文件中, 用来提高工作效率

例1: 编辑3-b3-vs-test.bat并运行

```
3-b3-vs-test.bat
文件 编辑 查看

@echo off
3-b3-2.exe < 3-b3-01.txt > 3-b3-01.out
txt_compare --file1 3-b3-01.out --file2 3-b3-01.std.out

3-b3-2.exe < 3-b3-02.txt > 3-b3-02.out
txt_compare --file1 3-b3-02.out --file2 3-b3-02.std.out

3-b3-2.exe < 3-b3-03.txt > 3-b3-03.out
txt_compare --file1 3-b3-03.out --file2 3-b3-03.std.out

3-b3-2.exe < 3-b3-04.txt > 3-b3-04.out
txt_compare --file1 3-b3-04.out --file2 3-b3-04.std.out
```

```
D:\25262-高程作业\x64\Debug>3-b3-vs-test.bat
文件相同.
文件相同.
文件相同.
文件相同.
```

例2: 编辑3-b3-gcc-test.bat并运行 (也可以合并到3-b3-vs-test.bat的后面)

● 观察bat中含中文且编码为UTF-8时的错误现象

```
3-b3-vs-test.bat 3-b3-gcc-test.bat
文件 编辑 查看

@echo off
D:\25262-高程作业\3-b3-2-C++方式分解浮点数\3-b3-2.exe < 3-b3-01.txt > 3-b3-01.gcc.out
txt_compare --file1 3-b3-01.gcc.out --file2 3-b3-01.std.out

D:\25262-高程作业\3-b3-2-C++方式分解浮点数\3-b3-2.exe < 3-b3-02.txt > 3-b3-02.gcc.out
txt_compare --file1 3-b3-02.gcc.out --file2 3-b3-02.std.out

D:\25262-高程作业\3-b3-2-C++方式分解浮点数\3-b3-2.exe < 3-b3-03.txt > 3-b3-03.gcc.out
txt_compare --file1 3-b3-03.gcc.out --file2 3-b3-03.std.out

D:\25262-高程作业\3-b3-2-C++方式分解浮点数\3-b3-2.exe < 3-b3-04.txt > 3-b3-04.gcc.out
txt_compare --file1 3-b3-04.gcc.out --file2 3-b3-04.std.out
```

```
D:\25262-高程作业\x64\Debug>3-b3-gcc-test.bat
文件相同.
文件相同.
文件相同.
文件相同.
```



§. 输入输出重定向及管道运算符

4. 用输入/输出重定向测试程序(以浮点数分解3-b3-2为例)

4.1. 准备测试数据

4.2 单数据调试

4.3 单数据测试并和demo对比运行结果

4.4 多数据测试并和demo对比运行结果

输入/输出重定向测试程序仍然存在的不足:

★ 每个数据一个数据文件，当测试点较多时，准备数据（虽然只做一次）仍然不方便

★ 如果数据需要修改，修改多个文件也不方便



§. 输入输出重定向及管道运算符

5. 用管道运算符测试程序

5.1. 管道运算符的基本使用

- ★ 管道运算符的作用：将前一个程序的输出当做后一个程序的输入
- ★ 可级联使用

例：两个程序，用管道运算符级联运算

程序1：输入两个正整数，输出最大值 (demo.exe)

程序2：输入一个正整数，输出其平方根 (demo2.exe)



§. 输入输出重定向及管道运算符

5. 用管道运算符测试程序

5.1. 管道运算符的基本使用

★ 管道运算符的作用：将前一个程序的输出当做后一个程序的输入

★ 可级联使用



例：两个程序，用管道运算符级联运算

程序1：输入两个正整数，输出最大值 (demo.exe)

程序2：输入一个正整数，输出其平方根 (demo2.exe)

```
#include <iostream>
using namespace std;

int main()
{
    int a, b;

    cin >> a >> b;
    cout << (a>b?a:b) << endl;

    return 0;
}
```

//注：去除了各种输入输出提示

```
#include <iostream>
using namespace std;

int main()
{
    int x;
    cout << "请输入一个整数" << endl;
    cin >> x;
    cout << "平方根是：" << sqrt(x) << endl;

    return 0;
}
```

```
D:\25262-高程作业\x64\Debug>notepad z.dat
```

```
D:\25262-高程作业\x64\Debug>type z.dat
10 15
```

```
D:\25262-高程作业\x64\Debug>type z.dat | demo
15
```

```
D:\25262-高程作业\x64\Debug>type z.dat | demo | demo2
请输入一个整数
平方根是： 3.87298
```

```
D:\25262-高程作业\x64\Debug>
```



§. 输入输出重定向及管道运算符

5. 用管道运算符测试程序

5.2. 用工具get_input_data.exe并配合管道运算符测试程序(以3-b3-2.exe为例)

- ★ 准备工作: 1、将get_input_data.exe放入D:\25262-高程作业\x64\debug下
2、用notepad编辑test-data.txt文件, 建立32个组,
将要求的32个数据分别放入, 每组一个数据

- 如果放在C:\Windows下, 则更通用
(放入指定目录并设置path是更好的方法, 但操作复杂, 不适合初学者)
- 认真阅读附件test-data.txt中的说明
- 允许新建, 使用其它文件名

- ★ 依次运行: 在cmd下用管道运算符依次运行, 观察结果(可放入bat中)

```
D:\25262-高程作业\x64\Debug>get_input_data test-data.txt 3-b3-01
9999999999.99

D:\25262-高程作业\x64\Debug>get_input_data test-data.txt 3-b3-01 | 3-b3-2.exe
请输入[0-100亿)之间的数字:
十亿位 : 9
亿位 : 9
千万位 : 9
百万位 : 9
十万位 : 9
万位 : 9
千位 : 9
百位 : 9
十位 : 9
圆 : 9
角 : 9
分 : 9

D:\25262-高程作业\x64\Debug>
```

管道运算符方式

test-data.txt

文件 编辑 查看

[3-b3-01]
9999999999.99

[3-b3-02]
2501200350.03

[3-b3-03]
803.03

[3-b3-04]
0.30

示例为4组,
其余略

- “get_input_data+管道运算符”相比输入重定向的优点:
- 1、所有数据(某题的若干数据, 若干题的若干数据)可以放在同一个文件中, 方便编辑和维护
 - 2、后期某些特殊情况, 需要用一个程序的输入做另一个程序的输出时, 可以级联



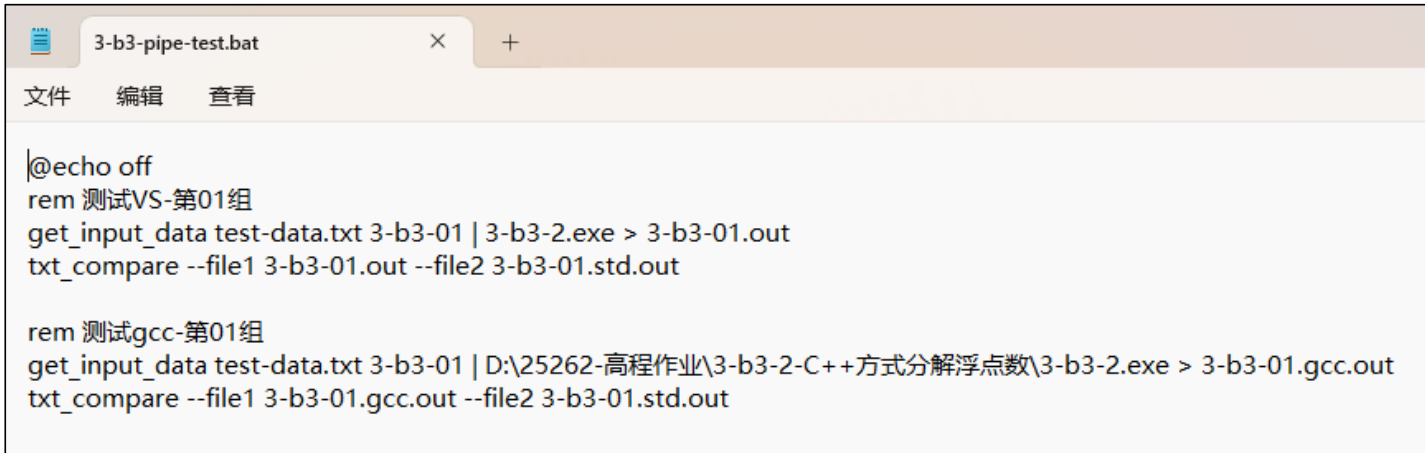
§. 输入输出重定向及管道运算符

5. 用管道运算符测试程序

5.2. 用工具get_input_data.exe并配合管道运算符测试程序(以3-b3为例)

★ get_input_data+管道运算符+输出重定向得到**输出文件**并进行txt_compare（允许多组）

```
D:\25262-高程作业\x64\Debug>notepad 3-b3-pipe-test.bat
```



```
@echo off
rem 测试VS-第01组
get_input_data test-data.txt 3-b3-01 | 3-b3-2.exe > 3-b3-01.out
txt_compare --file1 3-b3-01.out --file2 3-b3-01.std.out

rem 测试gcc-第01组
get_input_data test-data.txt 3-b3-01 | D:\25262-高程作业\3-b3-2-C++方式分解浮点数\3-b3-2.exe > 3-b3-01.gcc.out
txt_compare --file1 3-b3-01.gcc.out --file2 3-b3-01.std.out
```

```
D:\25262-高程作业\x64\Debug>3-b3-pipe-test.bat
文件相同.
文件相同.
```



§. 输入输出重定向及管道运算符

0. 第一次编程作业的不方便之处

- ★ 一个作业需要多次测试，每次Ctrl+F5运行后输入，**重复多次**
- ★ 如果作业测试过程中发现问题，修改后又得全部测试一遍，**重复多次**
- ★ 即使VS2026通过了，还得测试小熊猫，**重复多次**
- ★ 输出结果只能肉眼观察是否与demo匹配，**无法知道准确结果**

本文档解决的问题：**快速重复并准确知道结果正确与否**（红框4个痛点）

★ 不知道怎么设计测试数据

设计一个程序的测试数据（正常测试、边界测试、错误测试等）是思维缜密性训练的一个很重要的环节，也是将来从事工程/科研的基本素质，**必须自行培养**